

Why 400 tests?

Written by [Matthew](#) at [OceanApocalypseStudios](#) on July 14, 2026.

It will take about 1:45 to read this article at 220 WPM (*average reading speed*).

Before we get into the article, I want to inform you it will be much shorter than the last one, mostly because, currently, the focus is finishing [#60](#) and finally moving on to RSML's lexer and respective modularity system.

Today, we have officially hit the following milestone in RSML: 400 test methods... for a single class... and it's still not fully tested. Before you bring your anti-testing and `Console.WriteLine()` pitchforks forward, let us explain why we have implemented that many test methods...

Context matters

In this case, context matters a lot. The class that has received this much testing attention is [ReadOnlyStringBuffer](#), and it serves as the simplest buffer available in the entirety of the official RSML API, yet 400 tests were only half* of what was needed to get it to work properly.

At OceanApocalypseStudios, we believe testing buffers and big language components (such as lexers and parsers) is imperative to ease the task of getting them to work correctly. If this means testing a lot, then it's a sacrifice that we believe is worth making.

It's worth mentioning this does **not** mean in any way that we believe testing is only relevant when the components are critical: testing is still important when they're not — the only difference is that it's even more important when the components are critical.

Following the same rationale, we are **not** saying quantity matters more than quality: RSML could have 9000 tests and could be a broken mess or have 10 tests and work perfectly. We believe a good combination of both is the best compromise a developer can choose.

*This is an assumption, based on the current amount of test methods and the amount of methods that are yet to be tested.

Expecting more tests?

One word: **yes**. Not because we love writing tests, but because there are still methods that are publicly exposed yet are fully untested by now. Hopefully, they won't stay that way for much longer, and I'll be happy to write a new article once we can finally mark [#60](#) as closed.

Like I said, this was quite a small article, and a bigger one is to be expected soon.

Thank you for dropping by and reading! ❤️

Hello, DocFX!

Written by [Matthew](#) at [OceanApocalypseStudios](#) on July 10, 2026.

It will take about 1:57 to read this article at 220 WPM (*average reading speed*).

This is the very first blog article to be available to the public in RSML's site. Those who have followed the development of this project certainly have noticed the site is looking *somewhat* different. Well, you're clearly not wrong!

Today marks the day we, at OceanApocalypseStudios, have officially ditched [mkdocs](#) for [DocFX](#). Throughout this article, we'll be focusing on what aspects of Red Sea Markup Language lead to this decision and what it affects.

Why change?

The main reason for the change was convenience. We're a small team and, because of that, it takes some time to release the projects and it also takes time to write documentation for them, such as guides and walkthroughs, as well as some minor articles for those following RSML's development.

Back when we were using mkdocs, we needed to manually create the API reference: it was not fun. We ended up having the [XML documentation](#) in the source code and then we had to essentially port it manually to Markdown — it felt like a huge waste of time that could have been put into creating walkthroughs or expanding RSML's online presence.

DocFX, unlike mkdocs, is made **for** .NET primarily, so it automatically creates API references from the source projects, allowing us to focus on the more important tasks, which are the ones that require manual and granular control.

We want to clarify we are not saying mkdocs is not great: it is! However, for our case (a .NET library with XML comments), there was a better alternative, one that reads those very same XML comments and, using the built DLL, generates meaningful documentation from that.

Red Sea Markup Language

Search

OceanApocalypseStudios...
v2.0.0 ⭐ 4 🗒 0

[Home](#)
[Getting Started](#)
[Language](#)
[API](#)
[CLI](#)
[Blog](#)

API

Guides
Easy
Usage Demos

C#
Reference
RSML
RSML.Exceptions
RSML.Parser

Python
Reference

Home > API > C# > Reference

API Reference (C#)

On this page
Overview

The API reference for [RSML](#) in C# is organized by **namespace** and then **class**.

Overview

- [RSML](#)
 - [RSDocument](#)
 - `EvaluateDocument()`
 - `EvaluateDocument(string)`
 - `EvaluateDocument(bool, string?)`
 - `EvaluateDocument(string, string?)`
 - `EvaluateDocument(string, bool, string?)`
 - `LoadRSMLFromFile(string)`
 - `LoadRSMLFromFileAsync(string)`
 - `LoadRSMLFromFileIntoDocument(string)`
 - `NewFromFile(string)`
 - `RSDocument(string)`
 - `RSDocument(StringReader)`

Red Sea Markup Language

Search

[Introduction](#)
[Changelog](#)
[Language](#)
[API](#)
[Tools](#)
[Guides and Tutorials](#)

Filter by title

[OceanApocalypseStudios.RSML.Cache](#)
[OceanApocalypseStudios.RSML.Exceptions](#)
[OceanApocalypseStudios.RSML.Language.Lexer](#)
[OceanApocalypseStudios.RSML.Language.Syntax](#)
[OceanApocalypseStudios.RSML.Native](#)
[OceanApocalypseStudios.RSML.Sdk](#)
[OceanApocalypseStudios.RSML.Sources](#)
[IBufferReader](#)
[IBuffer<TItem>](#)
[ISource](#)

[API / OceanApocalypseStudios.RSML.Sources / ReadOnlyStringBuffer / Methods / TryGetWord](#)

Edit this page

Method TryGetWord

TryGetWord(int, Span<char>, out bool, out int)

Tries to read the word at index `index`, which can be a span of whitespace (if the starting index points to whitespace) or a span of non-whitespace content (if the starting index does not point to whitespace).
[View Source](#) | [Edit this page](#)

Declaration

```
public bool TryGetWord(int index, Span<char> destination, out bool isWhitespace, out int itemCount)
```

Parameters

TYPE	NAME	DESCRIPTION
int	index	The index at which the word starts.
Span<char>	destination	The span which will be the destination for the read content.
bool	isWhitespace	True if the span is fully comprised of whitespace. If False, no whitespace is present.
int	itemCount	The amount of characters that were written to <code>destination</code> .

Returns

TYPE	DESCRIPTION
bool	False if the buffer is out of bounds or an exception occurred.

Implements

[IBuffer<TItem>.TryGetWord\(int, Span<TItem>, out bool, out int\)](#)

Remarks

Unlike with other [ReadOnlyStringBuffer](#) methods, this one does not accept the EOF index (index at `Length`), because it is not considered part of any word.

The old and new RSML websites, side-by-side

Is the site fully finished?

Nowhere near finished ☹

We are still working on v3.0.0-prerelease1, which means the site will also be updated frequently with new documentation, guides and whatnot. Even if we pretend for a second no more documentation will be added (which is unthinkable, as the site still has pages marked with “TODO”), we will change some aspects of the site's design.

We, at OceanApocalypseStudios, are constantly working on both new and existing projects: we've released [MurkyMarshParser](#) to the public very recently and are now focused on crafting the biggest RSML release so far, little by little.

The move from mkdocs to DocFX has proved to complete a lot of tasks that were previously ours to get done, giving us time to focus on what truly matters.

Hello, DocFX!

Written by [Matthew](#) at [OceanApocalypseStudios](#) on July 10, 2026.

It will take about 1:57 to read this article at 220 WPM (*average reading speed*).

This is the very first blog article to be available to the public in RSML's site. Those who have followed the development of this project certainly have noticed the site is looking *somewhat* different. Well, you're clearly not wrong!

Today marks the day we, at OceanApocalypseStudios, have officially ditched [mkdocs](#) for [DocFX](#). Throughout this article, we'll be focusing on what aspects of Red Sea Markup Language lead to this decision and what it affects.

Why change?

The main reason for the change was convenience. We're a small team and, because of that, it takes some time to release the projects and it also takes time to write documentation for them, such as guides and walkthroughs, as well as some minor articles for those following RSML's development.

Back when we were using mkdocs, we needed to manually create the API reference: it was not fun. We ended up having the [XML documentation](#) in the source code and then we had to essentially port it manually to Markdown — it felt like a huge waste of time that could have been put into creating walkthroughs or expanding RSML's online presence.

DocFX, unlike mkdocs, is made **for** .NET primarily, so it automatically creates API references from the source projects, allowing us to focus on the more important tasks, which are the ones that require manual and granular control.

We want to clarify we are not saying mkdocs is not great: it is! However, for our case (a .NET library with XML comments), there was a better alternative, one that reads those very same XML comments and, using the built DLL, generates meaningful documentation from that.

Red Sea Markup Language

Search

OceanApocalypseStudios...

v2.0.0
☆ 4
👤 0

Home
Getting Started
Language
API
CLI
Blog

API
Guides
Easy
Usage Demos
C#
Reference
RSML
RSML.Exceptions
RSML.Parser
Python
Reference

Home > API > C# > Reference

On this page

Overview

API Reference (C#)

The API reference for [RSML](#) in C# is organized by **namespace** and then **class**.

Overview

- [RSML](#)
 - [RSDocument](#)
 - `EvaluateDocument()`
 - `EvaluateDocument(string)`
 - `EvaluateDocument(bool, string?)`
 - `EvaluateDocument(string, string?)`
 - `EvaluateDocument(string, bool, string?)`
 - `LoadRSMLFromFile(string)`
 - `LoadRSMLFromFileAsync(string)`
 - `LoadRSMLFromFileIntoDocument(string)`
 - `NewFromFile(string)`
 - `RSDocument(string)`
 - `RSDocument(StringReader)`

Red Sea Markup Language

Search

Introduction
Changelog
Language
API
Tools
Guides and Tutorials

Filter by title

OceanApocalypseStudios.RSML.Cache
OceanApocalypseStudios.RSML.Exceptions
OceanApocalypseStudios.RSML.Language.Lexer
OceanApocalypseStudios.RSML.Language.Syntax
OceanApocalypseStudios.RSML.Native
OceanApocalypseStudios.RSML.Sdk
OceanApocalypseStudios.RSML.Sources
IBufferReader
IBuffer<TItem>
ISource

API / OceanApocalypseStudios.RSML.Sources / ReadOnlyStringBuffer / Methods / TryGetWord

Edit this page

Method TryGetWord

`TryGetWord(int, Span<char>, out bool, out int)`

Tries to read the word at index `index`, which can be a span of whitespace (if the starting index points to whitespace) or a span of non-whitespace content (if the starting index does not point to whitespace). [View Source](#) | [Edit this page](#)

Declaration

```
public bool TryGetWord(int index, Span<char> destination, out bool isWhitespace, out int itemCount)
```

Parameters

TYPE	NAME	DESCRIPTION
<code>int</code>	<code>index</code>	The index at which the word starts.
<code>Span<char></code>	<code>destination</code>	The span which will be the destination for the read content.
<code>bool</code>	<code>isWhitespace</code>	True if the span is fully comprised of whitespace. If False, no whitespace is present.
<code>int</code>	<code>itemCount</code>	The amount of characters that were written to <code>destination</code> .

Returns

TYPE	DESCRIPTION
<code>bool</code>	False if the buffer is out of bounds or an exception occurred.

Implements

`IBuffer<TItem>.TryGetWord(int, Span<TItem>, out bool, out int)`

Remarks

Unlike with other [ReadOnlyStringBuffer](#) methods, this one does not accept the EOF index (index at `Length`), because it is not considered part of any word.

The old and new RSML websites, side-by-side

Is the site fully finished?

Nowhere near finished 😞

We are still working on `v3.0.0-prerelease1`, which means the site will also be updated frequently with new documentation, guides and whatnot. Even if we pretend for a second no more documentation will be added (which is unthinkable, as the site still has pages marked with “TODO”), we will change some aspects of the site's design.

We, at OceanApocalypseStudios, are constantly working on both new and existing projects: we've released [MurkyMarshParser](#) to the public very recently and are now focused on crafting the biggest RSML release so far, little by little.

The move from mkdocs to DocFX has proved to complete a lot of tasks that were previously ours to get done, giving us time to focus on what truly matters.

Why 400 tests?

Written by [Matthew](#) at [OceanApocalypseStudios](#) on July 14, 2026.

It will take about 1:45 to read this article at 220 WPM (*average reading speed*).

Before we get into the article, I want to inform you it will be much shorter than the last one, mostly because, currently, the focus is finishing [#60](#) and finally moving on to RSML's lexer and respective modularity system.

Today, we have officially hit the following milestone in RSML: 400 test methods... for a single class... and it's still not fully tested. Before you bring your anti-testing and `Console.WriteLine()` pitchforks forward, let us explain why we have implemented that many test methods...

Context matters

In this case, context matters a lot. The class that has received this much testing attention is [ReadOnlyStringBuffer](#), and it serves as the simplest buffer available in the entirety of the official RSML API, yet 400 tests were only half* of what was needed to get it to work properly.

At OceanApocalypseStudios, we believe testing buffers and big language components (such as lexers and parsers) is imperative to ease the task of getting them to work correctly. If this means testing a lot, then it's a sacrifice that we believe is worth making.

It's worth mentioning this does **not** mean in any way that we believe testing is only relevant when the components are critical: testing is still important when they're not — the only difference is that it's even more important when the components are critical.

Following the same rationale, we are **not** saying quantity matters more than quality: RSML could have 9000 tests and could be a broken mess or have 10 tests and work perfectly. We believe a good combination of both is the best compromise a developer can choose.

*This is an assumption, based on the current amount of test methods and the amount of methods that are yet to be tested.

Expecting more tests?

One word: **yes**. Not because we love writing tests, but because there are still methods that are publicly exposed yet are fully untested by now. Hopefully, they won't stay that way for much longer, and I'll be happy to write a new article once we can finally mark [#60](#) as closed.

Like I said, this was quite a small article, and a bigger one is to be expected soon.

Thank you for dropping by and reading! ❤️